

In the Name of God



Course Project: Introduction to Machine Learning

Phase 2

OOD Detection and Generation with NADE

Instructor

Dr. S. Amini

Department of Electrical Engineering
Sharif University of Technology

Fall 1404

Deadline:

24 Bahman, 1404 at 23:59

Authors:

Arshak Razvani, Mohammad Eshtehardian

Contents

Contents

1	Important Notes	1
2	Introduction: From GMMs to Autoregressive Models	2
3	Theoretical Background	2
3.1	The Chain Rule of Probability	2
3.2	NADE Architecture	2
4	Theoretical Questions	3
4.1	Question 1: Parameter Efficiency	3
4.2	Question 2: The Computational “Trick”	3
4.3	Question 3: Maximum Likelihood and Loss Function	3
4.4	Question 4: Sampling vs. Scoring	4
5	Practical Implementation & Experiments	4
5.1	Dataset Preparation	4
5.2	Task 1: Implement NADE	4
5.3	Task 2: Training	4
5.4	Task 3: OOD Detection	5
5.5	Task 4: Generation	5
5.6	Important Implementation Notes	5

1 Important Notes

Please carefully read the following points:

1. The two phases of this project will contribute a total of **4 points** to your final course grade.
2. After the final submission, there will be an in-person project defense session where you must present your code and outputs to the teaching assistants and answer their questions. **All group members must be fully familiar with every part of the project.** A single score will be assigned to the entire group.
3. All sections of both phases are mandatory. There will be no optional or bonus parts.
4. The Phase 2 submission deadline is a **hard deadline** and is on **24 Bahman**. **Late submission days cannot be used for Phase 2.**
5. All simulations must be implemented using Python. You are allowed to use any libraries you have used in the assignments, such as **numpy**, **scipy**, and **pytorch**. However, **you must implement the algorithms yourself** (e.g., the NADE forward pass and sampling), and you may not use libraries that already provide the algorithm implementations.
6. The project deliverables consist of both your code and a written report. The report should include answers to the questions, plots, figures, and conclusions. In the end, one **.zip** file containing all code files, and a **.pdf** report must be uploaded to the CW platform. Only one group member needs to submit the project.
7. If you use any external sources (books, articles, websites, etc.) to answer the questions, make sure to cite them properly.
8. **Any form of plagiarism will result in a zero for both parties involved.**
9. For any questions, please contact the project instructors via Telegram.

Good luck!

2 Introduction: From GMMs to Autoregressive Models

In Phase 1, we modeled the In-Distribution (ID) data using a Gaussian Mixture Model (GMM). While GMMs are powerful, they have limitations in high-dimensional spaces (like images) and often struggle to capture complex, non-linear dependencies between pixels.

In **Phase 2**, we explore a more modern approach: **Autoregressive Models**. Specifically, we will implement **NADE (Neural Autoregressive Distribution Estimation)**. Unlike GMMs, which model the joint density $p(\mathbf{x})$ directly via latent clusters, autoregressive models decompose the joint density into a product of conditional densities using the probability chain rule.

This phase has three main goals:

1. **Theoretical Understanding:** Deriving the mechanics of NADE and its training objective.
2. **Implementation:** Building a NADE from scratch for Binarized MNIST.
3. **Application:** Using the trained model for **OOD Detection** and **Image Generation**.

3 Theoretical Background

3.1 The Chain Rule of Probability

For a binary input vector $\mathbf{x} \in \{0, 1\}^D$, the joint probability distribution $p(\mathbf{x})$ can be exactly factored as:

$$p(\mathbf{x}) = \prod_{d=1}^D p(x_d \mid \mathbf{x}_{<d}) \quad (1)$$

where $\mathbf{x}_{<d} = \{x_1, \dots, x_{d-1}\}$ denotes the dimension preceding d .

3.2 NADE Architecture

NADE parameterizes these conditionals using a neural network with shared weights to improve parameter efficiency and computational speed. For the d -th dimension, the probability that $x_d = 1$ is modeled as:

$$\mathbf{h}_d = \sigma(\mathbf{W}_{\cdot, <d} \mathbf{x}_{<d} + \mathbf{c}) \quad (2)$$

$$\hat{x}_d = p(x_d = 1 \mid \mathbf{x}_{<d}) = \sigma(\mathbf{V}_d^\top \mathbf{h}_d + b_d) \quad (3)$$

Where:

- σ is the sigmoid activation function.
- $\mathbf{W} \in \mathbb{R}^{H \times D}$ and $\mathbf{V} \in \mathbb{R}^{D \times H}$ are weight matrices.
- $\mathbf{c} \in \mathbb{R}^H$ and $\mathbf{b} \in \mathbb{R}^D$ are biases.
- $\mathbf{W}_{\cdot, <d}$ denotes the first $d - 1$ columns of $\mathbf{W}$.

The key efficiency of NADE is that the hidden state $\mathbf{h}_d$ can be computed recursively (or cumulatively) rather than re-computing the matrix multiplication for every step.

4 Theoretical Questions

4.1 Question 1: Parameter Efficiency

Consider a binary image with D pixels.

1. If we modeled $p(\mathbf{x})$ using a fully expressive tabular method (assigning a probability to every possible state), how many parameters would we need?
2. If we use NADE with hidden size H , how many trainable parameters (weights and biases) does the model have?
3. Compare the scaling of these two approaches with respect to D .

4.2 Question 2: The Computational “Trick”

One of the main contributions of NADE is utilizing weight sharing to speed up the computation of the conditionals.

1. Let $\mathbf{a}_d = \mathbf{W}_{\cdot, < d} \mathbf{x}_{< d} + \mathbf{c}$ be the pre-activation of the hidden layer for the d -th conditional. Write an equation to compute $\mathbf{a}_{d+1}$ using $\mathbf{a}_d$ and the input x_d .
2. Based on this recursive update, what is the computational complexity (Big-O notation) of performing a single forward pass to compute $p(\mathbf{x})$ for one image? Compare this to the complexity if we did *not* use the recursive update and treated each conditional as a separate neural network.

4.3 Question 3: Maximum Likelihood and Loss Function

We wish to train the model by minimizing the negative log-likelihood (NLL) of the training data.

1. Show that minimizing the negative log-likelihood of the joint distribution $p(\mathbf{x})$ is equivalent to minimizing the sum of Binary Cross Entropy (BCE) losses for each individual pixel d :

$$\mathcal{L}(\mathbf{x}) = - \sum_{d=1}^D [x_d \log \hat{x}_d + (1 - x_d) \log(1 - \hat{x}_d)] \quad (4)$$

2. Why do we typically **binarize** the MNIST dataset (setting pixel values > 0.5 to 1, else 0) before training a standard NADE? What would happen if we used continuous values $[0, 1]$ with the formulation above?

4.4 Question 4: Sampling vs. Scoring

1. Describe the algorithm to **sample** (generate) a new image $\mathbf{x}_{\text{new}}$ from a trained NADE model.
2. Explain why calculating the probability $p(\mathbf{x})$ of an existing image is computationally faster (and can be parallelized) than sampling a new image, which must be strictly sequential.

5 Practical Implementation & Experiments

5.1 Dataset Preparation

- Load the MNIST dataset.
- **Binarize** the data: Set pixel values > 0.5 to 1, and others to 0.
- **Split by Class:**
 - **In-Distribution (ID) / Train:** Select all images corresponding to digits **0, 1, 2, 3, 4**. Use 80% for training and 20% for validation.
 - **Out-of-Distribution (OOD) / Test:** Select all images corresponding to digits **5, 6, 7, 8, 9**. These will *only* be used during the testing phase.

5.2 Task 1: Implement NADE

Implement a NADE class in PyTorch (or NumPy). Your class should include:

- `__init__`: Initialize weights $\mathbf{W}, \mathbf{V}$ and biases $\mathbf{c}, \mathbf{b}$.
- `forward(x)`: Compute $\log p(\mathbf{x})$ for a batch of input images. *Hint: Use the recursive/cumulative update rule derived in Question 2.*
- `sample(n_samples)`: Generate `n_samples` new images pixel-by-pixel.

5.3 Task 2: Training

- Train your NADE model on the **ID (0–4)** training set.
- Use a hidden dimension size H of at least 500.
- Use an optimizer like Adam.
- **Plot** the average training loss and validation loss over epochs.

5.4 Task 3: OOD Detection

Similar to Phase 1, we will use the log-likelihood as an anomaly score.

1. Compute the average log-likelihood assigned by your model to the **ID Test Set** (digits 0–4 held out from training).
2. Compute the average log-likelihood assigned by your model to the **OOD Test Set** (digits 5–9).
3. **Plot Histograms:** Overlay the histograms of the log-likelihoods for ID samples vs. OOD samples. Do they separate well?
4. **Metric:** Calculate the **AUROC** (Area Under the ROC Curve) for detecting OOD samples. (Label ID as 0 and OOD as 1, or vice versa).

5.5 Task 4: Generation

1. Use your `sample` function to generate 10 new images.
2. Visualize them in a grid.
3. Since the model was only trained on 0–4, do the generated images look like 0–4, or do they look like random noise? Discuss the quality of the generation.

5.6 Important Implementation Notes

- **Ordering:** For standard NADE, you can process pixels in raster order (row by row). You do not need random orderings.
- **Numerical Stability:** When computing binary cross entropy, use functions that take logits (e.g., `torch.nn.functional.binary_cross_entropy_with_logits`) or implement the log-sum-exp trick to avoid $\log(0)$.
- **Time Constraint:** Sampling is slow because it requires $D = 784$ passes through the network. Don't try to generate thousands of images; 10–20 is sufficient for visualization.